

EXP 3: Policy and Value Function Representation using Bellman Equations

Aim

To implement and understand policy and value function representation in Reinforcement Learning using the Bellman equations, and to calculate the value of different states under a given policy.

Algorithm

1. Initialize the states, actions, rewards, discount factor , and policy.
2. Select a policy that defines the action taken in each state.
3. Initialize the value function for all states.
4. Apply the Bellman expectation equation:
5. Update the value of each state repeatedly until the values converge.
6. Represent the final values as the value function of the given policy.
7. Compare the values of states to identify which states are more rewarding.

CODE

```
import numpy as np
import matplotlib.pyplot as plt
states = ['S1', 'S2', 'S3', 'Terminal']
rewards = [5, 2, 10, 0]
gamma = 0.9
V = np.zeros(len(states))
for iteration in range(20):
    new_V = V.copy()
    for i in range(len(states)-1):
        new_V[i] = rewards[i] + gamma * V[i+1]
    V = new_V
print("Final Value Function")
```

```
s,value in zip(states,V):  
    print(s,":", round(value,2))  
  
plt.figure(figsize=(6,4)) plt.bar(states,  
V)  
  
plt.title("State Value Function using Bellman Equation") plt.xlabel("States")  
plt.ylabel("Value")  
plt.grid(axis='y')  
plt.show()
```

OUTPUT

Final Value Function

S1: 14.9

S2:11.0

S3:10.0

Terminal:0.0



